

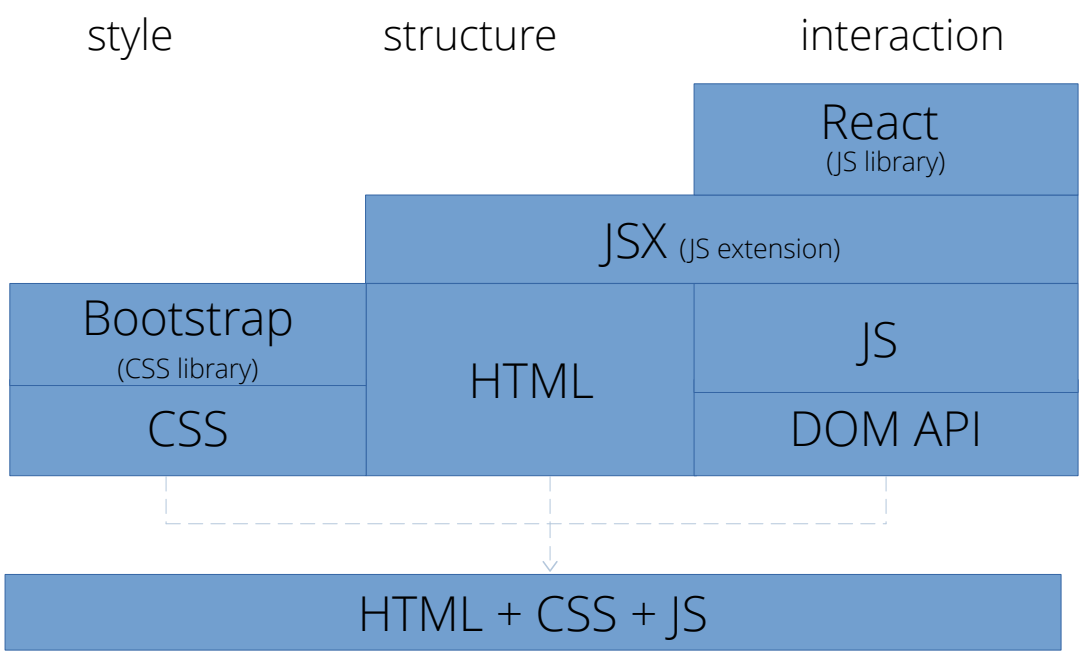


Course review



Course materials originally by Sonsoles López-Pernas, adapted by Maria Kuismin

Course technologies



collaboration
version control

GitHub

git

Three sections and states of Git

Working Directory	Staging Area	Local Repository	Remote Repository
Where we edit files	Where you prepare changes for the next commit.	The code that has been committed	The remote server for sharing and collaboration

---------- Three sections of a Git project ----------

A file in the working directory can be unmodified, modified, or staged. The staging area is where staged changes are recorded. The Git directory stores committed snapshots

Commit naming convention




```
➜ ng-poopy  master ✗  git add .
➜ ng-poopy  master ✗  git cz

All commit message lines will be cropped at 100 characters.

? Select the type of change that you're committing: (Use arrow keys)
❯ feat:     A new feature
  fix:      A bug fix
  docs:     Documentation only changes
  style:    Changes that do not affect the meaning of the code
            (white-space, formatting, missing semi-colons, etc)
  refactor: A code change that neither fixes a bug or adds a feature
  perf:     A code change that improves performance
  test:     Adding missing tests
  chore:    Changes to the build process or auxiliary tools
            and libraries such as documentation generation
```

Git and the platforms

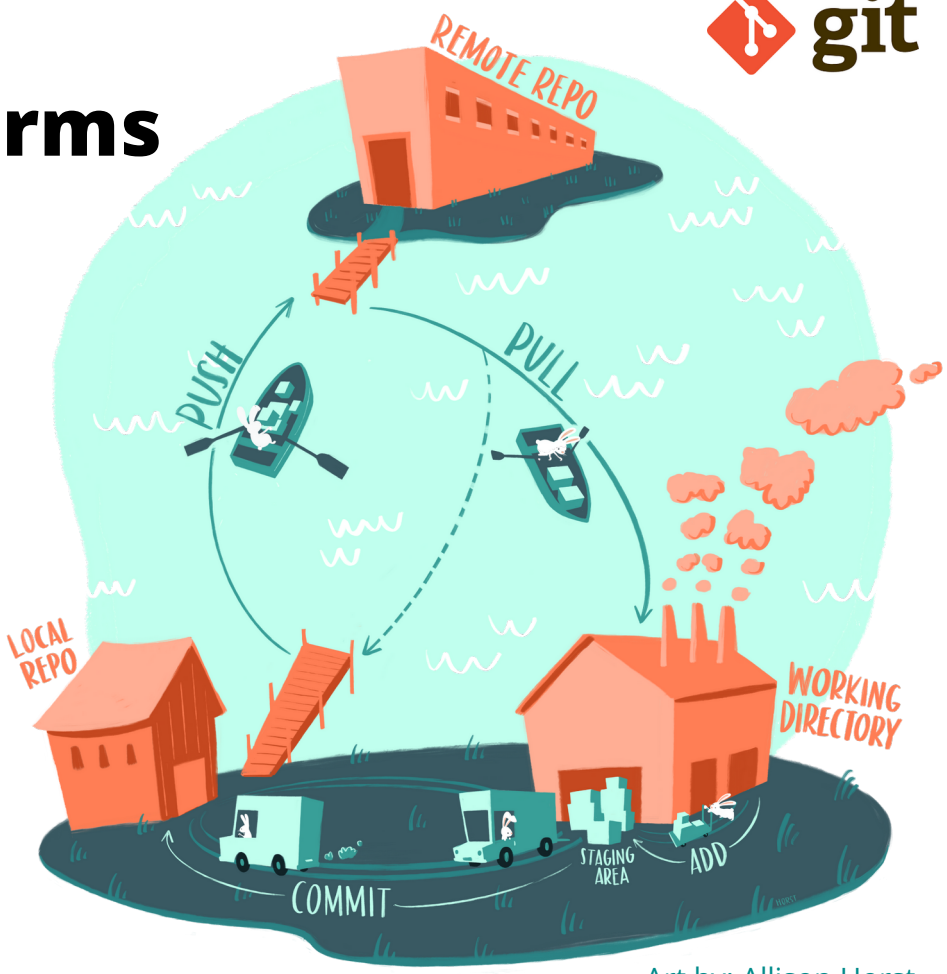
- git – version control locally
- GitHub – a cloud-based platform for hosting and collaboration

BitBucket https://bitbucket.org

Codeberg https://codeberg.org/

GitHub https://github.com

GitLab https://about.gitlab.com



Art by: Allison Horst

Working Directory | Staging Area | Local Repository | Remote Repository

Clone vs. Fork – the difference

- `clone` copies an existing repo to your own computer, makes a local copy
- Use clone when you want to
  - Get a local working copy of a repository
  - Work on a repo where you already have access
  - Download your own repo to your computer
  - e.g. you only want to use/build/install the project locally, not contribute back.

- `fork` makes a remote copy of someone's repo under your own GitHub account
- Use fork when
  - You dont permission to push directly to the original repo
  - You want your own writable copy on GitHub
  - You want to contribute changes back using a pull request

fork + clone : You almost always fork first, then clone your fork when contributing to someone else's project



Copy-paste is not prompt engineering

Effective prompting requires understanding what you're building.

Common patterns that create problems:
- Pasting the whole assignment and asking for a solution
- Accepting code you don't understand
- Applying changes without checking what they do
- Assuming the first answer is the right one

Common patterns that work better:
- Break the problem into small steps
- Ask about concepts you don't understand
- Ask for specific changes for the existing code you shared
- Ask AI to explain a solution and concider different options before applying one

The difference isn't about "knowing everything" — it's about being able to make informed decisions about what AI suggests.



The role of your understanding

You can get AI generated code for some app in seconds.
You can also get one from a tutorial, a template, or a colleague.

The question is not where the code came from. But what happens when
- the app breaks and no one wrote the code
- a user reports a bug and you need to fix it
- the requirements change and you need to adapt
- the code becomes yours to maintain
- something goes wrong and someone asks who is accountable

AI-generated code is like any other code you didn't write yourself: someone has to understand it before it can be trusted, changed, or explained.



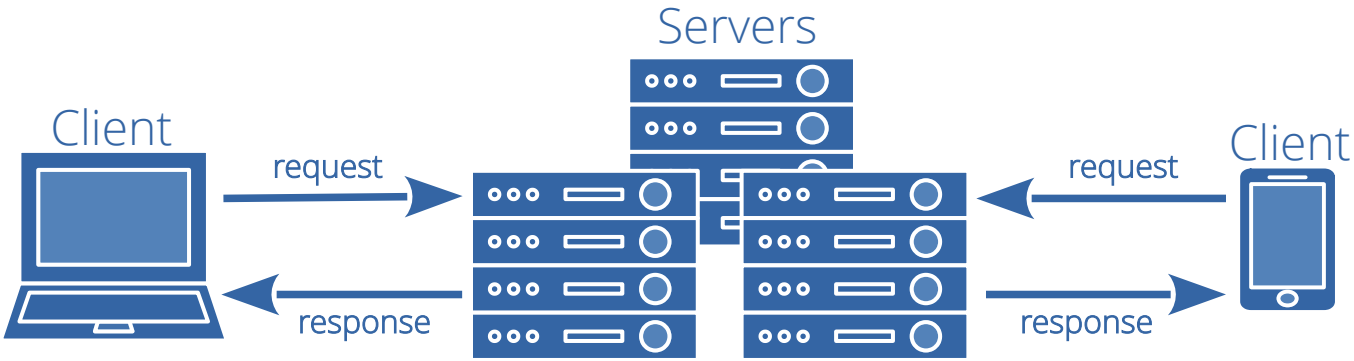
Custom agents in AI coding tools

Coding tools allow you to configure how the AI responds to you in a specific project.

You can define what role the AI plays (assistant, tutor, reviewer), what it should and shouldn't do and how it should respond.

This makes AI coding tools customizable to your needs.

Client-server architecture



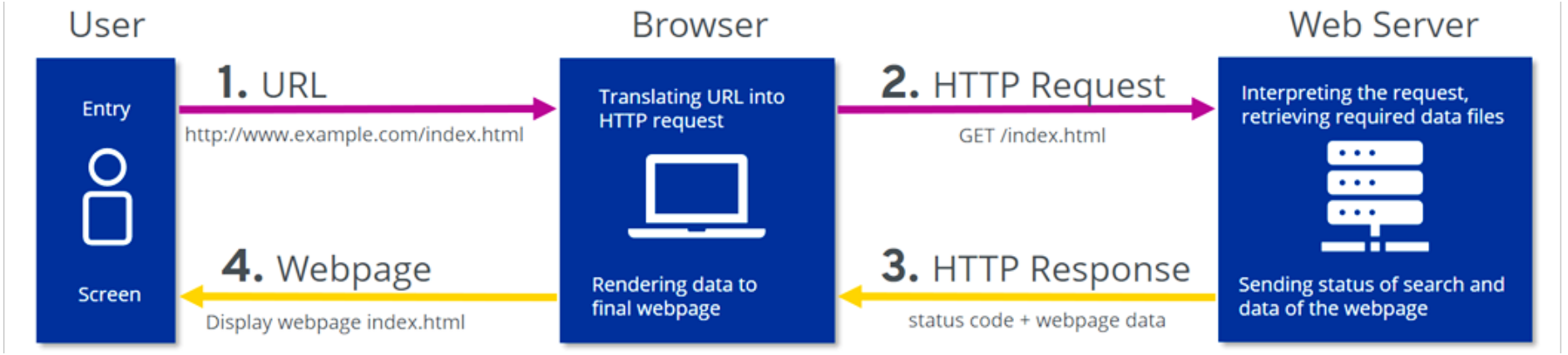
The client makes request for service to server.
The service responds to the request.

Hypertext Transfer Protocol

- supports hypermedia access to Web pages and other Internet services
- is a stateless transactional request-response protocol

HTTP transaction is always initiated from the client side

- client establishes HTTP connection with the server or reuses an existing one, and sends HTTP Requests to the server over the HTTP connection
- server answers the client by sending HTTP Responses over the connection
- request and response are independent, though related, transmissions





HTTP methods

The type of primitive of an HTTP request is indicated by the method or verbs:

- Uniform interface, methods allow you to manage resources on the server with the four CRUD primitives:
  - **POST**: Create a resource on the server.
  - **GET**: Read a resource from the server.
  - **PUT**: Update a resource from the server.
  - **DELETE**: Delete a resource from the server.



HTTP Responses

Informative Responses (1xx)

- 100 Continue: Continue partial request

Request completed (2xx)

- 200 OK: Operation completed successfully, resource served
- 201 Created: Resource created or updated successfully

Redirect (3xx)

- 301 Moved Permanently: Resource on another server
- 304 Not Modified: When the client already has the data

Client Error (4xx)

- 400 Bad Request: Incorrect command sent
- 404 Not Found: Resource not found in repository
- 405 Method Not Allowed: Method not supported by server
- 409 Conflict: Conflict with the resource status
- 410 Gone: Resource no longer exists

Server Error (5xx)

- 500 Internal Server Error: The server has errors

Block and line elements

HTML elements can be displayed as block or inline by default.

Block: Block elements take up the entire width of the page and force a new line before and after, e.g.,<article> <aside> <canvas> <footer> <h1> <h2> <h3> <h4> <h5> <h6> <header> <p> <pre> <section> .
The element: A block element that is used to contain other HTML elements inside it. It has no meaning, it is semantically neutral.

Line: Inline elements only take up the width needed and do not force new lines, e.g. , <a>
 <img> <button> <input> <select> <textarea>...
The element: A line element that is used to contain other HTML elements inside it. It has no meaning, it is semantically neutral.

Semantic vs. physical emphasis

Semantic emphasizers:

```
<em>emphasized text</em>
<strong> important text</strong>
<code> source code</code>
<dfn>definition</dfn>
<cite>title of a work</cite>
```

Physical emphasizers:

```
<small> smaller text </small>
<b>bold text</b>
<i>Italic text</i>
<u>underlined text</u>
<s>strikethrough text</s>
text<sub>subscript</sub>
text<sup>superscript</sup>
```

emphasized text
important text
`source code`
definition
title of a work
<small>smaller text</small>
bold text
Italic text
<u>underlined text</u>
~~strikethrough text~~
text$_{\text{subscript}}$
text^{superscript}

What is cascading?

Cascade refers to something that flows or occurs in a series of stages, originally a sequence of waterfalls

A cascade is a chain of events or a flowing sequence.

Cascading in Cascading Style Sheet = which style rule apply when multiple rules target the same element

- multiple style declarations “flow down” from different sources, and is resolved into a single result



Pixabay: Peter Schmidt

Three levels of cascading

1. Origin and importance

`!important` > inline > internal > external > browser default

Styles marked with `!important` gets the highest priority

2. Specificity - hierarchy of CSS selectors

3. Source order - order in which a CSS rules are defined

the rule defined later takes the priority

Pixabay: Peter Schmidt

Specificity Hierarchy

inline > id > class > elemenet

Every CSS selector has its place in the specificity hierarchy.

1. Inline styles – e.g.: <h1 style="color: pink;">
2. IDs – e.g : #navbar
3. Classes, pseudo-classes, attribute selectors – e.g. : .test, :hover, [href]
4. Elements and pseudo-elements – e.g. : h1, ::before

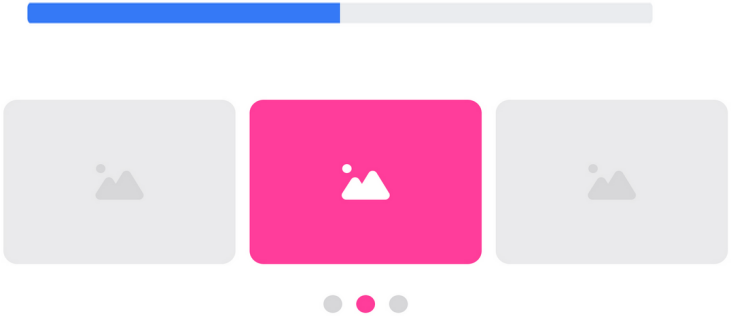
There's numerical value for specificity, each ID value, class value etc. are calculated

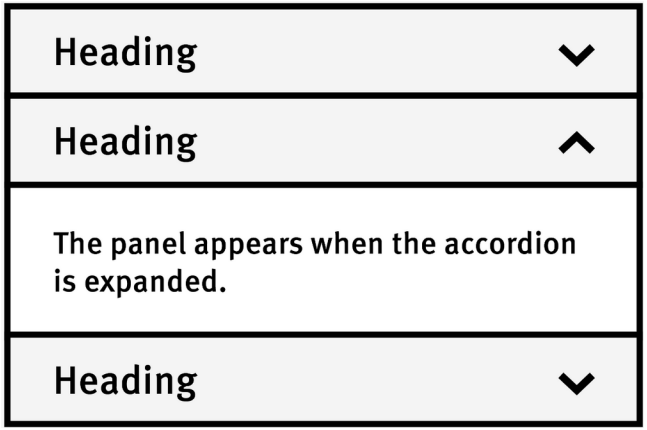
Key principles of accessible design

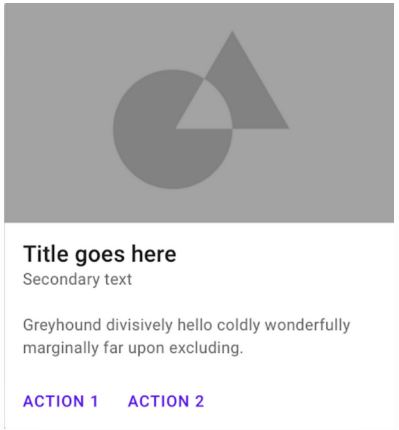
- **Perceivable** – information must be visible or audible.
- **Operable** – users can navigate with mouse, keyboard, or assistive tech.
- **Understandable** – content and navigation are clear.
- **Robust** – works well with different devices and assistive technologies.

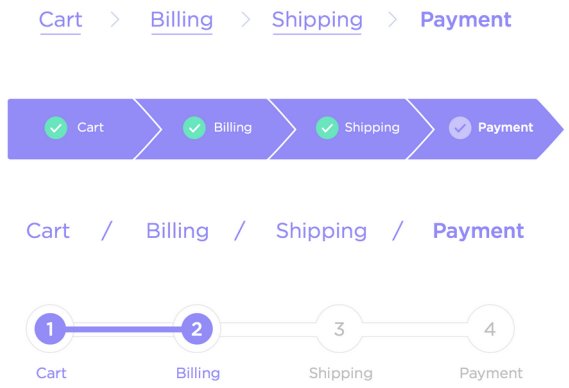
Web User Interface components

- UI elements are the building blocks of apps and web sites.
- They are what users interact with when they are using the product.

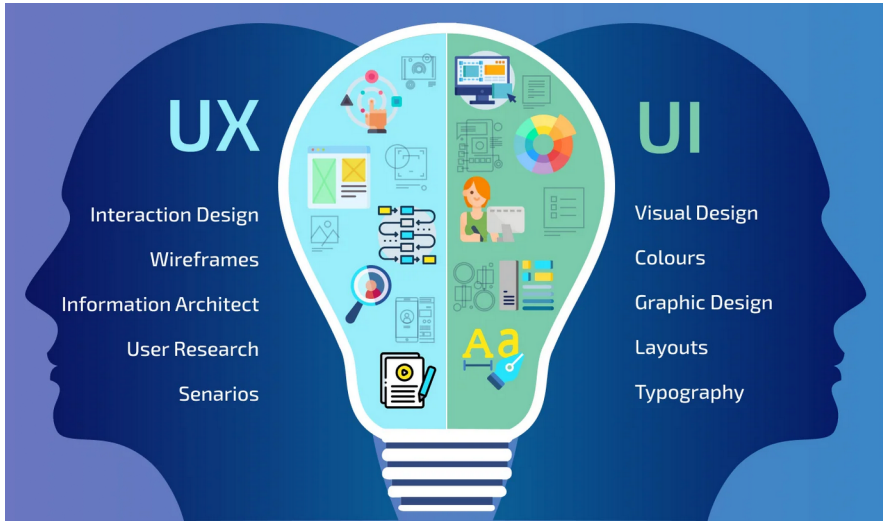








UI & UX

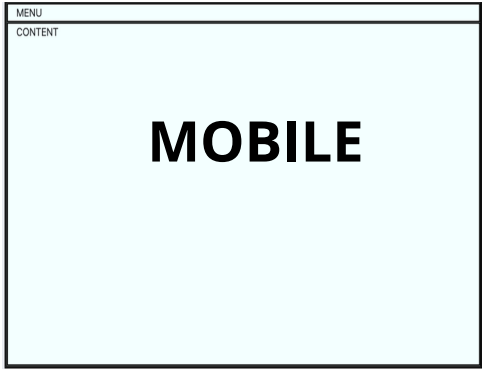


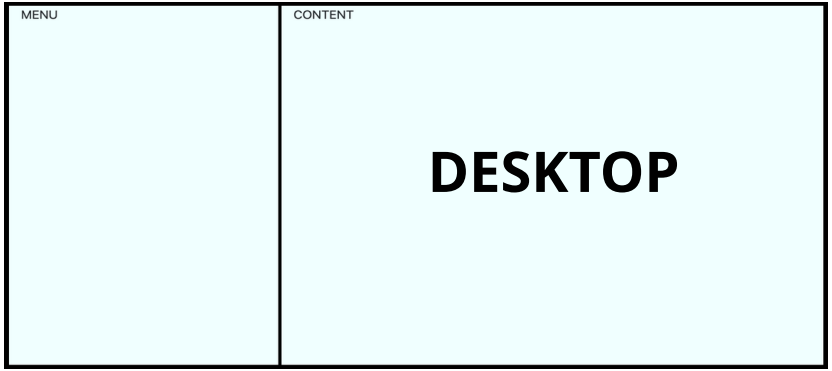
- UX, User eXperience
  = how users experience a product
- Familiar design patterns build user trust, breaking patterns confuses users
- Designers reuse well-known UI components to keep experiences predictable
- This is why the same components appear across many websites

Layout - desktop and mobile

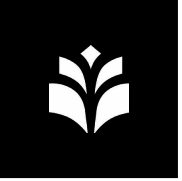
```
<div class="container-fluid">
    <div class="row">
        <div class="col-12 col-lg-4">MENU</div>
        <div class="col-12 col-lg-8">CONTENT</div>
    </div>

</div>
```





https://appbrewery.github.io/bootstrap-layout



JS

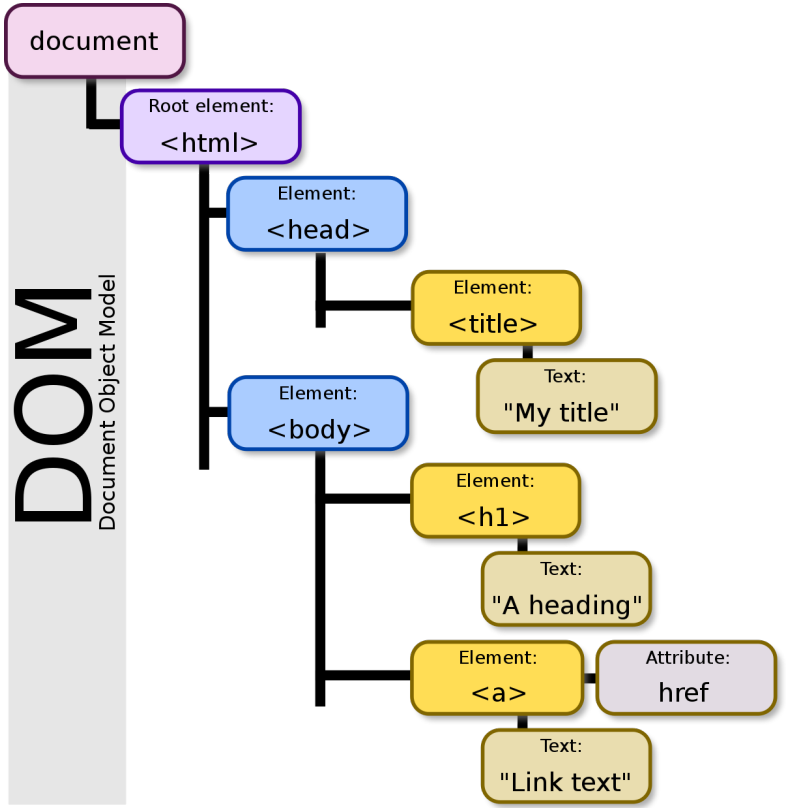
What JavaScript can do in the browser

The browser gives JavaScript access to several APIs:

- DOM (Document Object Model) — modify the HTML page (reads, changes, creates, and removes HTML elements on the page) → focus of this course
- other browser APIs: Graphics (Canvas, WebGL), Audio/Video, Geolocation, Storage, Notifications, ...

Document Object Model

DOM (Document Object Model) is the JS API provided by the browser to manipulate the page's HTML



Manipulating DOM Elements ›
Event listeners

Events are one of the most important aspects of the DOM, allowing you to make your web pages interactive. Common events include clicks, key presses, form submissions, etc.

The element that we interact with

The type of interaction

```
document.getElementById('myButton').addEventListener('click', function() {
  alert('Button was clicked!');});
```

What happens after this interaction

https://www.w3schools.com/jsref/dom_obj_event.asp

Async/await – the modern standard

Built on top of Promises — reads like regular sequential code.

Rules:

→ define the function with async

→ use await before the slow operation

→ wrap in try/catch for error handling (same as Java)

→ the rest of the page continues normally while waiting

```
async function getCats() {
  try {
    const response = await
fetch('https://api.example.com/cats');
    const cats = await response.json();
    console.log(cats);  // use the data here
  } catch (error) {
    console.error('Something went wrong:',
error);
  }
}
getCats();  // call the function
```

REST APIs

A REST API is an address (URL) that another server provides.
(Representational State Transfer Application Programming Interface)

How it works:
1. Our JavaScript sends an HTTP request to that URL
2. The server processes the request
3. The server responds with data in JSON format
4. We use the data to update the DOM

Connection to what you already know:
- HTTP requests: GET, POST, PUT, DELETE
- JSON format: { "name": "Toast", "age": 6, "likes": 47 }
- Fetching data uses async/await

→ because waiting for a server response takes time

Server remembers nothing between requests

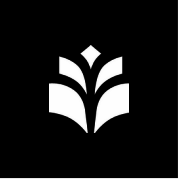
Statelessness

Each request from a client to the server must be independent and contain all the necessary information for the server to fulfill that request.

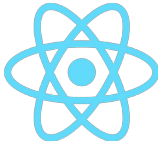
The server does not remember previous requests or store session information.

in practice

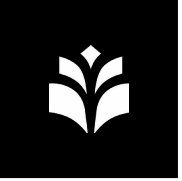
→ every request you send must include authentication (if needed)
→ the server treats each request independently
→ this is why APIs often require an API key in every request

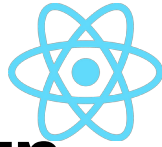


UI as reusable components



- solution: split the UI into independent, reusable parts
- Each component:
  → is devoted to one specific task
  → receives data as input (props)
  → returns a user interface
- Direction web development has taken:
  Web Components, Angular, React, Vue.js
- In JavaScript it is very easy to create spaghetti
  — better to structure applications in modules



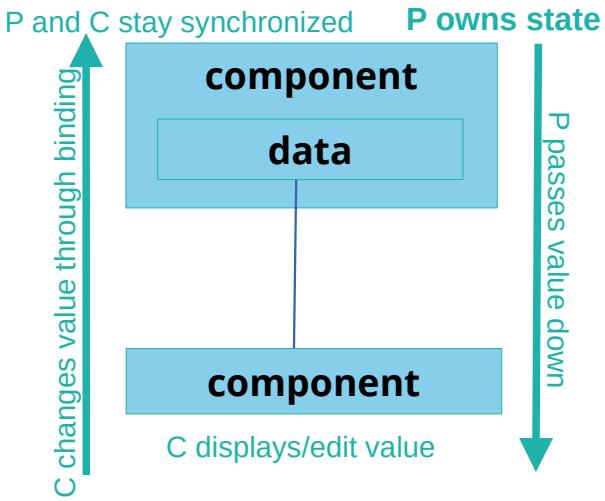


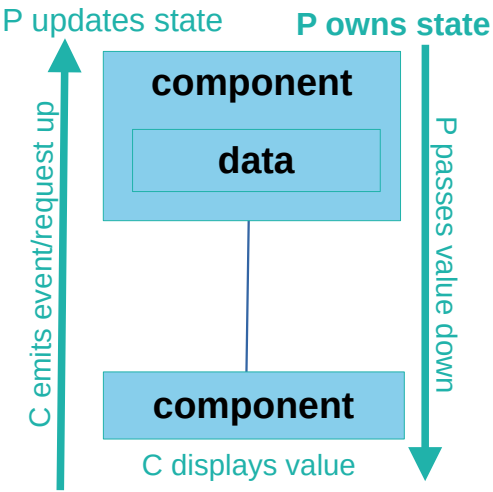
From requirements to component design

Process:
1. Read the requirements
2. Find repeating patterns → component candidate
3. Identify what data each part needs → props candidates
4. Identify what changes over time → state candidates
5. Draw the component tree

Ask yourself:
→ What repeats? (e.g., each post looks the same)
→ What changes? (e.g., like count updates)
→ What is separate? (e.g., adding a post is its own feature)
→ Who needs this data? (decides where state lives)

Data binding — two approaches





Two-way data binding – data flows in both directions
→ Application updates UI
→ UI automatically updates application data
→ used by Vue (when using v-model). Angular

One-way data binding – data flows in one direction only
→ Application updates UI
→ UI cannot directly update application data
→ used by Vue (via props), React

Both solve the same problem: keeping UI in sync with data. Parent owns state in both cases; the difference is whether the child can only request changes or can sync changes back through binding.

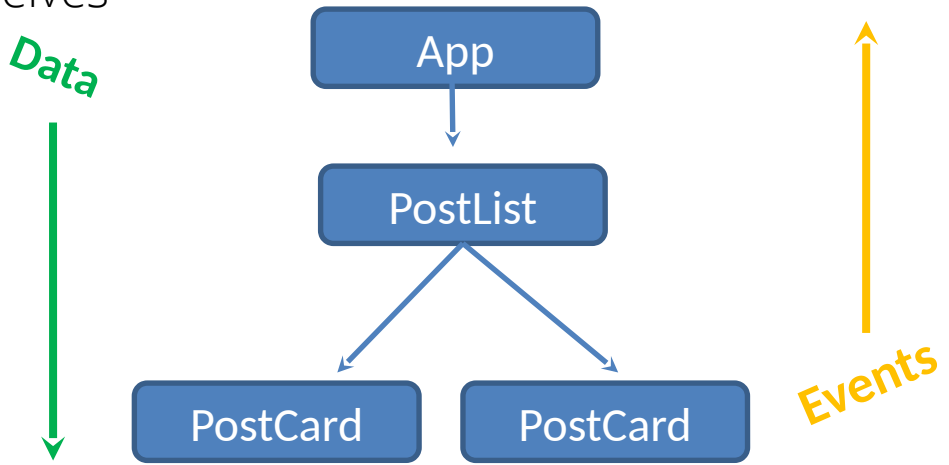
Data flows one way — predictable

In one-way data binding data flows down, events flow up
→ predictable, easier to debug?

Data (props) flow from parent to child: App → PostList → PostCard.
Parent decides what data each child receives

Events flow from child to parent:
PostCard → PostList → App.

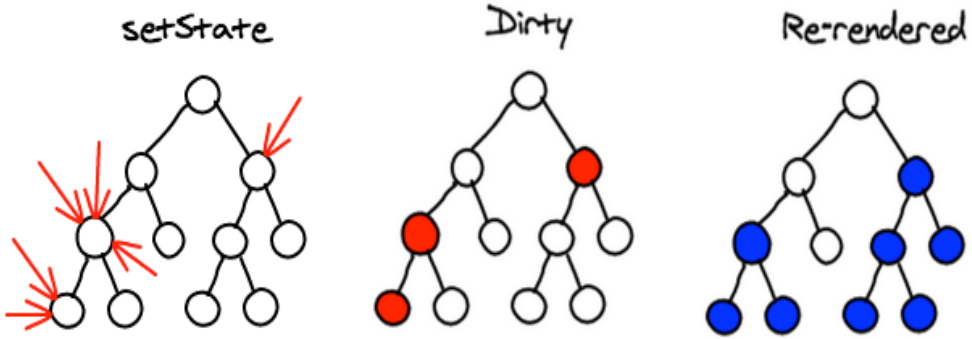
Child notifies parent when something happens (e.g., "user clicked like")
— the parent decides what changes.



Virtual DOM — efficient UI updates

Without React (vanilla JS) developer decides manually which DOM element to update → Works for simple apps, but becomes error-prone as complexity grows

With React developer describes what the UI should look like. React compares new vs. previous Virtual DOM (diffing)React automatically updates only the necessary parts



→ developer does not need to track which element changed

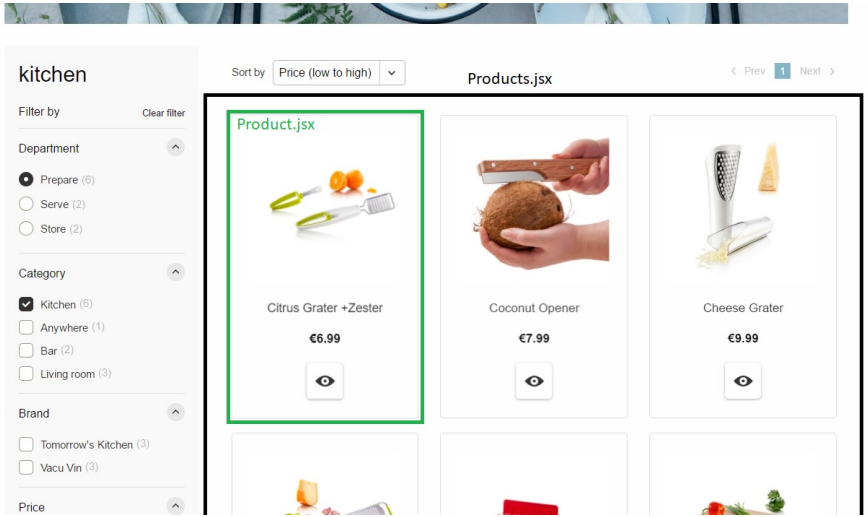
A component as a UI building block

In React, a component

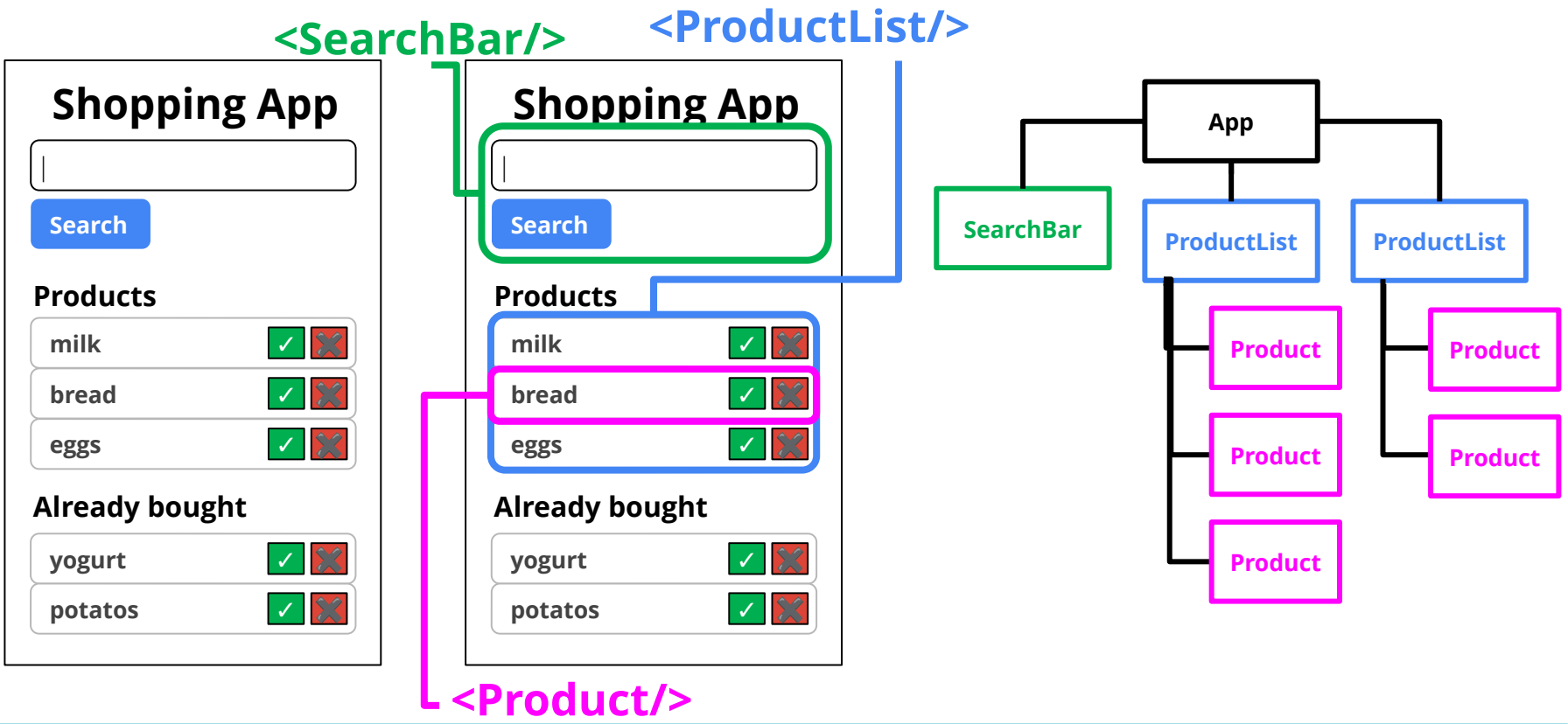
→ is a function that returns the UI as JSX (JavaScript XML)

→ receives data as input (props)

→ is independent and reusable



Example: Shopping App



Component — function returning JSX

A component is a function that
→ accepts props as input
→ returns a React element (JSX)

Both traditional and arrow function syntax work, still arrow functions are more common in modern React.

```
// Traditional function syntax
function Welcome(props) {
  return <h1>Hello, {props.name}
</h1>;
}

// Arrow function syntax
const Welcome = (props) => {
  return <h1>Hello, {props.name}
</h1>;
};
```

Babel transpiles to function Welcome(props) {

```
  return React.createElement(
    'h1',          // element type
    null,          // props (here null)
    'Hello, ',     // child 1 — text
    props.name     // child 2 — variable
  );
}
```


Note: Class components also exist but are no longer recommended.

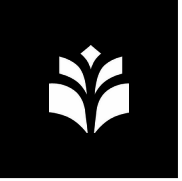
State must be updated via setter function

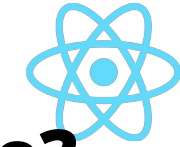
```
const [count, setCount] = useState(0);
```

Incorrect: `count = count + 1;`
JavaScript variable changes, but React does not know about it
→ UI never updates

Correct: `setCount(count + 1);`
React stores the new value → React schedules a re-render → component re-renders
→ UI shows the new value

React tracks state changes only through the setter function.
Direct modification is invisible to React.





Which component should own the state?

Rule: state belongs to the lowest common ancestor of all components that need it.

Ask:

Does only one component need this data? → State belongs in that component (component state)

Do multiple components need the same data? → State belongs in their common parent (lift state up)

Kažigram example:

posts list – needed by PostList AND PostForm → belongs in App (common parent)

form input text – needed only by PostForm while typing → belongs in PostForm (component state)

like count – could belong in LikeButton or Post (if needed only by it), or App if persisted

Lifting state up

Problem: PostList and PostForm both need the posts array.
Solution: move the state up to their common parent — App.
- PostForm calls props.onAddPost(newPost) ← event flows up
- App updates state → re-render → PostList gets new posts

```
function App() {
  const [posts, setPosts] = useState([]);          ← state in App

  const addPost = (newPost) => {                   ← function in App
    setPosts([...posts, newPost]);
  };
  return (
    <div>
      <PostList posts={posts} />                   ← data flows down
      <PostForm onAddPost={addPost} />             ← function flows down
    </div>
  );
}
```

Events update state — React re-renders

The cycle:

1. User clicks the button
2. onClick calls setCount(count + 1)
3. React stores the new value
4. React re-renders the component
5. UI shows the updated count

UI = f(state)
You describe what the UI looks like for any given state. React handles when and what to update.

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

useEffect: for side effects

Side effect = anything that reaches outside React's rendering, e.g. changing the browser (document.title, localStorage), setting up a timer or subscription, or fetching data from a server (example on the slide)

```
function PostList() {
  const [posts, setPosts] = useState([]);  // state for fetched data
                                           // starts as empty array
  useEffect(() => {   // runs after render
    const getData = async () => {
      const res = await fetch('https://api.example.com/posts');
                          // reaches OUTSIDE React o an external server
      const data = await res.json();
      setPosts(data);   // brings data back in, triggers re-render
    };
    getData();
  }, []);             // [] = run only once when component appears
  return posts.map(post =>
    <PostCard key={post.id} title={post.title} />
  );
}
```

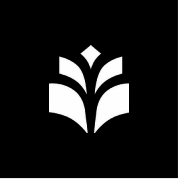
useEffect + fetch — where and why

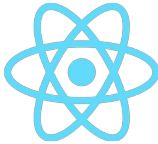
In vanilla JS: fetch anywhere — click handler, page load
In React: fetch belongs inside useEffect

Why? The component function runs on every render. Fetch directly in component body leads to infinite loop: fetch → setProducts(data) → re-render → fetch again → ...

useEffect with empty dependency array breaks the loop: runs exactly once, when component first appears.

```
useEffect(() => {
    // fetch goes here – inside useEffect
    // not directly in the component body
}, []);
```





Multipage experience with routing

- routing allows users to have a multi-page experience in SPAs while staying within one HTML document
- client side routing allows app to update the URL from a link click without making another request for another document from the server. Instead, app can immediately render some new UI and make data requests with fetch to update the page with new information.
- enables faster user experiences because the browser doesn't need to request an entirely new document or re-evaluate CSS and JS assets for the next page

Organizing a larger React app

As your app grows, all files in src/ becomes hard to manage. Recommended structure:

```
src/
   components/          ← reusable components
      PostCard.jsx
      Feed.jsx
      PostForm.jsx
      Header.jsx
   pages/               ← one file per route
      Feed.jsx     ← route: "/"
      NewPost.jsx      ← route: "/new"
   App.jsx              ← routes defined here
   main.jsx             ← entry point
```

pages/ → one component per route

components/ → everything used in multiple pages

How does it all fit together

SETUP CHAIN

Node.js — runtime environment that runs JavaScript outside the browser

↓ (comes with npm)

npm — package manager that installs libraries (React, Vite, etc.)

↓ (installs)

Vite — development server with hot reloading + production build tool

↓ (uses Babel internally)

React + Babel

DEVELOPMENT CHAIN

WE WRITE React components in JSX (.jsx files)

↓

Vite serves the file

↓

Babel transpiles JSX → vanilla JavaScript

↓

BROWSER RUNS vanilla JavaScript

DEPLOYMENT CHAIN

WE WROTE React components in JSX (.jsx files)

↓

npm run build (one time)

↓

Vite + Babel transpile everything once

↓

Static files generated: HTML, CSS, JS + assets

↓

Upload to server

↓

BROWSER RUNS vanilla JavaScript (no Vite, no Babel needed)

How JSX becomes JavaScript

WE WRITE JSX (.jsx files)

↓

Vite serves the file

↓

Babel transpiles JSX to JavaScript

↓

BROWSER RUNS JavaScript

JSX transpiles to React.createElement()...

we write JSX

```
function Hello() {                §JS
  return                          §REACT
    <span>Hello!</span>           §HTML
  ;                               §JS
}                                 §JS
```

Babel outputs JavaScript

```
function Hello() {                §JS
  return                          §JS
    React.createElement(          §REACT
      'span',                     §JS
      null,                       §JS
      'Hello!'                    §JS
    );                            §JS
}                                 §JS
```

JSX is an extension to JS syntax, JS+XML.

React is a JS library.

JSX is a convenience for the developer, the actual runtime code is plain JavaScript that calls React's functions.

```
JavaScript
React
HTML/JSX
```

...so React code IS JavaScript

Greeting.jsx

```
function Greeting (props) {
    const message = "Welcome to React";
  return (
      <div>
        <h1>Hello, { props.name }!</h1>
        <p>{ message }</p>
      </div>
  );
 }
export default Greeting;
```

React code IS JavaScript.
React is a JavaScript library.
React component is a JavaScript function.

JavaScript
React
HTML/JSX

What changed, what stayed the same?

	JavaScript	React
How you update UI	You make DOM changes directly with code	Your code produces a representation of the UI, React uses it to update the DOM
DOM manipulation	You do it manually (createElement, appendChild)	React does it for you (virtual DOM diffing)
State	You track it yourself (variables)	useState tracks it and triggers re-render
Event handling	AddEventListener separately	onClick, onSubmit as JSX props
What stayed the same?	.map(), .filter(), spread, ternary, arrow functions, template literals, fetch, async/await	